UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 19

Proyectos de múltiples archivos

→ NOTAS DE CLASE

Índice

1 1976: el bug que no era un bug **2**

2 El problema: cuando un archivo no alcanza **2**

3 Archivos de cabecera: el contrato entre módulos **2**
3.1 Declaración versus definición 2
3.2 Guards de inclusión 3
3.3 <> vs. "" 3

4 Compilación por etapas **4**
4.1 Los dos pasos del compilador 4
4.2 Comandos individuales 4
4.3 Una línea de compilación que no para de crecer 4

5 Make: automatizar la compilación **5**
5.1 Un primer Makefile: solo la línea de compilación 5
5.2 Variables: el compilador y sus flags 5
5.3 Variables: la lista de archivos y el nombre del ejecutable 6
5.4 Dependencias y archivos objeto 6
5.5 Cómo decide Make qué recompilar 7
5.6 Regla patrón: evitar la repetición 7

6 CMake: multiplataforma y sin repetición **8**
6.1 ¿Por qué CMake si ya tenemos Make? 8
6.2 Un `CMakeLists.txt` mínimo 8
6.3 Flujo de trabajo 9

7 Proyecto ejemplo: sistema de personas **9**

8 Ejercicios **10**

1976: el bug que no era un bug

En 1976, Stuart Feldman trabajaba en los Laboratorios Bell cuando un colega pasó horas depurando un programa que juró haber corregido. El problema no estaba en el código: el ejecutable había sido compilado desde una versión *antigua* del fuente. Alguien había editado el archivo, pero nadie había vuelto a compilar.

Feldman decidió que las computadoras debían rastrear qué archivos habían cambiado y recompilar solo lo necesario. En semanas escribió **Make**, la herramienta que desde entonces acompaña a todos los proyectos de software en Unix. Casi cincuenta años después, todavía la usamos.

Stuart Feldman inventó Make en 1976 en Bell Labs. Su diseño refleja la filosofía Unix: una sola herramienta que hace una cosa bien y se compone con otras.

Décadas después surgieron alternativas: **CMake** (1999) para proyectos multiplataforma, **Ninja** (2012) para velocidad extrema en proyectos grandes (Chromium, LLVM), y **Bazel** (2015, Google) para repositorios monolíticos de millones de líneas.

Referencia: Feldman, S. I. (1979). “Make — A Program for Maintaining Computer Programs.” *Software: Practice and Experience*, 9(4), 255–265.

El problema: cuando un archivo no alcanza

Empieza un proyecto con 50 líneas. Un mes después tiene 500; un semestre después, 5 000. Editar y recompilar un solo archivo gigante tiene varios costos:

- **Lectura:** buscar una función entre miles de líneas es lento.
- **Colaboración:** dos personas no pueden editar el mismo archivo sin conflictos de versión.
- **Compilación:** cambiar una función obliga a recompilar todo el programa, aunque el cambio no afecte al resto.
- **Reutilización:** no puedes usar tus funciones en otro proyecto sin copiar el archivo completo.

La solución es dividir el código en **módulos**: unidades independientes que se pueden escribir, compilar y reutilizar por separado.

Archivos de cabecera: el contrato entre módulos

- **Declaración versus definición**

Cada módulo existe en dos archivos:

`modulo.h` Contiene las **declaraciones**: le dice al compilador qué funciones y tipos *existen* y cuál es su forma. No hay código ejecutable.

`modulo.cpp` Contiene las **definiciones**: cómo funcionan esas funciones. Incluye su propio `.h` para que el compilador verifique que la definición coincide con la declaración.

funciones.h — solo declaraciones:

```
void saludar(char*);
int  sumar(int, int);
int  restar(int, int);
```

funciones.cpp — definiciones (incluye su propio `.h`):

```
#include "funciones.h"
void saludar(char* n) { cout << "Hola " << n << endl; }
int  sumar(int a, int b) { return a + b; }
int  restar(int a, int b) { return a - b; }
```

La regla de C++ que gobierna esto se llama **ODR** (*One Definition Rule*): puedes *declarar* algo cuantas veces quieras, pero solo puedes *definirlo* una vez en todo el programa.

• Guards de inclusión

¿Qué pasa si `main.cpp` incluye `funciones.h` directamente **y** también incluye `auxiliar.h`, que a su vez incluye `funciones.h`? El preprocesador copiaría el contenido de `funciones.h` dos veces, produciendo un error de redefinición.

Los **guards de inclusión** lo evitan:

```
#ifndef FUNCIONES_H   // Si la macro no existe...
#define FUNCIONES_H   // ...crearla (primera inclusión)

void saludar(char*);
int  sumar(int, int);
int  restar(int, int);

#endif                // Fin del bloque protegido
```

La segunda vez que el preprocesador encuentra `#include "funciones.h"`, la macro `FUNCIONES_H` ya existe y salta todo el contenido.

> La alternativa moderna es escribir `#pragma once` como primera línea del `.h`. Todos los compiladores actuales la soportan y es más simple que el guard manual. Ambas formas son correctas.

• <> vs. ""

Sintaxis	Busca en	Usar para
`#include <vector>`	Rutas del sistema	Librería estándar y paquetes instalados
`#include "funciones.h"`	Directorio actual primero	Tus propios archivos

Regla simple: si el archivo lo escribiste tú, usa `""`; si viene con el compilador o una librería externa, usa <>.

Compilación por etapas

• Los dos pasos del compilador

Cuando escribes `g++ main.cpp funciones.cpp -o programa`, en realidad ocurren dos pasos distintos:

Paso	Entrada	Salida
1. Compilar	`main.cpp` `funciones.cpp`	`main.o` `funciones.o`
2. Enlazar	`main.o` + `funciones.o`	`programa`

El **compilador** traduce cada `.cpp` a código máquina de forma independiente, produciendo un **archivo objeto** (`.o`). El **enlazador** (*linker*) conecta esos objetos: cuando `main.o` llama a `saludar`, el linker busca en `funciones.o` dónde está esa función y resuelve la referencia.

• Comandos individuales

```
# Paso 1: compilar cada .cpp por separado (genera .o, no enlaza)
g++ -c funciones.cpp -o funciones.o
g++ -c main.cpp      -o main.o

# Paso 2: enlazar todos los .o en un ejecutable
g++ main.o funciones.o -o programa
```

✔ Recuerda

La bandera `-c` le dice a `g++` que solo compile (no enlace). Sin `-c`, intenta compilar y enlazar todo lo que reciba.

• Una línea de compilación que no para de crecer

Un proyecto de dos archivos es manejable. Pero imagina el proyecto de personas que veremos al final de esta sesión: seis módulos, cada uno con su `.h` y su `.cpp`. La línea de compilación queda así:

```
g++ -std=c++17 -Wall -Wextra -o programa \
    main.cpp auxiliar.cpp obtener_datos.cpp \
    mate.cpp buscar.cpp
```

Y eso es solo al inicio. Cuando el proyecto crezca y necesites flags adicionales, rutas de include o librerías externas:

```
g++ -std=c++17 -Wall -Wextra -O2 -I./include \
    -I./lib/json/include \
    main.cpp auxiliar.cpp obtener_datos.cpp    \
    mate.cpp buscar.cpp graficas.cpp reportes.cpp \
    -L./lib -ljson -lm -o programa
```

Cada vez que cambies un archivo tienes que recordar y volver a escribir todo eso. Cada vez que agregues un `.cpp` tienes que buscarlo en la línea y añadirlo en el lugar correcto.

Hay una solución mejor.

Make: automatizar la compilación

• Un primer Makefile: solo la línea de compilación

Un `Makefile` es un archivo de texto que le dice a Make qué comandos ejecutar para construir tu programa. La forma más básica es simplemente envolver la línea de compilación en una **regla**:

```
programa:
 g++ -std=c++17 -Wall -Wextra -o programa \
     main.cpp auxiliar.cpp obtener_datos.cpp \
     mate.cpp buscar.cpp
```

Con esto, en lugar de recordar y teclear la línea larga, solo escribes:

```
make
```

Make busca el archivo llamado `Makefile` en el directorio actual, lee la primera regla y ejecuta el comando.

> El comando dentro de una regla debe comenzar con un carácter de **tabulación** (Tab), no con espacios. Es el error más común al escribir Makefiles: la mayoría de editores muestran ambos igual, pero Make los distingue y falla si encuentra espacios.

La estructura de cualquier regla es siempre la misma:

```
objetivo: dependencias
<TAB>   comando para construir el objetivo
```

objetivo es el nombre del archivo que se quiere generar. **dependencias** son los archivos que necesita para generarlo. Por ahora las dejamos vacías; las completaremos pronto.

• Variables: el compilador y sus flags

Repetir `g++` y las flags en cada regla es un problema: si cambias de compilador o decides agregar un flag nuevo, tienes que modificar cada línea. Las **variables** de Make resuelven esto.

Primero extraemos el compilador:

```
CXX = g++

programa:
 $(CXX) -std=c++17 -Wall -Wextra -o programa \
     main.cpp auxiliar.cpp obtener_datos.cpp \
     mate.cpp buscar.cpp
```

Luego los flags:

```
CXX      = g++
CXXFLAGS = -std=c++17 -Wall -Wextra

programa:
 $(CXX) $(CXXFLAGS) -o programa \
     main.cpp auxiliar.cpp obtener_datos.cpp \
     mate.cpp buscar.cpp
```

Para usar el valor de una variable se escribe `$(NOMBRE)`. Make sustituye el valor antes de ejecutar el comando.

• Variables: la lista de archivos y el nombre del ejecutable

El mismo principio aplica a la lista de fuentes y al nombre del ejecutable:

```
CXX      = g++
CXXFLAGS = -std=c++17 -Wall -Wextra
TARGET   = programa
SRCS     = main.cpp auxiliar.cpp obtener_datos.cpp \
           mate.cpp buscar.cpp

$(TARGET): $(SRCS)
 $(CXX) $(CXXFLAGS) $(SRCS) -o $(TARGET)
```

Ahora agregar un módulo nuevo es editar solo la variable `SRCS`. Cambiar el nombre del ejecutable es editar solo `TARGET`.

El proyecto crece y necesitas un módulo `graficas.cpp`. Con el Makefile anterior solo cambias una línea:

```
SRCS = main.cpp auxiliar.cpp obtener_datos.cpp \
       mate.cpp buscar.cpp graficas.cpp
```

La próxima vez que corras `make`, el comando completo ya incluirá `graficas.cpp` automáticamente.

• Dependencias y archivos objeto

El Makefile anterior todavía tiene un problema: recompila *todos* los fuentes cada vez que cambias cualquier cosa. Si modificas solo `buscar.cpp`, Make vuelve a compilar `main.cpp`, `auxiliar.cpp` y los demás aunque no los tocaste.

La solución es declarar cada `.o` como objetivo separado, con sus propias dependencias:

```
CXX      = g++
CXXFLAGS = -std=c++17 -Wall -Wextra
TARGET   = programa
OBJS     = main.o auxiliar.o obtener_datos.o mate.o buscar.o
```

```
$(TARGET): $(OBJS)
 $(CXX) $(OBJS) -o $(TARGET)

main.o: main.cpp auxiliar.h obtener_datos.h mate.h buscar.h
 $(CXX) $(CXXFLAGS) -c main.cpp

auxiliar.o: auxiliar.cpp auxiliar.h persona.h
 $(CXX) $(CXXFLAGS) -c auxiliar.cpp

obtener_datos.o: obtener_datos.cpp obtener_datos.h persona.h auxiliar.h
 $(CXX) $(CXXFLAGS) -c obtener_datos.cpp

mate.o: mate.cpp mate.h persona.h
 $(CXX) $(CXXFLAGS) -c mate.cpp

buscar.o: buscar.cpp buscar.h persona.h
 $(CXX) $(CXXFLAGS) -c buscar.cpp

clean:
 rm -f $(OBJS) $(TARGET)
```

Las dependencias de cada `.o` incluyen los `.h` que ese módulo utiliza, no solo su propio `.cpp`. Si cambias `persona.h` (por ejemplo, agregas un campo), Make recompila todos los módulos que dependen de él.

• Cómo decide Make qué recompilar

Make compara la **fecha de modificación** de cada objetivo con la de sus dependencias. Si alguna dependencia es más reciente que el objetivo, lo reconstruye; si no, no hace nada.

```
# Solo cambias buscar.cpp:
$ make
g++ -std=c++17 -Wall -Wextra -c buscar.cpp      # solo este
g++ main.o auxiliar.o obtener_datos.o mate.o \
    buscar.o -o programa                         # reenlaza
```

`main.o`, `auxiliar.o` y los demás no se tocan: sus fechas son más recientes que sus fuentes. En proyectos con cientos de archivos, esto reduce el tiempo de compilación de minutos a segundos.

• Regla patrón: evitar la repetición

Escribir una regla distinta para cada `.cpp` es tedioso. Una **regla patrón** lo generaliza:

```
# Cualquier .o se construye desde el .cpp con el mismo nombre
%.o: %.cpp
 $(CXX) $(CXXFLAGS) -c $< -o $@
```

Las variables automáticas `$<` y `$@` son especiales de Make:

Variable	Significado
`$@`	El objetivo actual (`buscar.o`)
`$<`	La primera dependencia (`buscar.cpp`)
`$^`	Todas las dependencias

Con la regla patrón, el Makefile completo queda mucho más corto:

```
CXX      = g++
CXXFLAGS = -std=c++17 -Wall -Wextra
TARGET   = programa
SRCS     = main.cpp auxiliar.cpp obtener_datos.cpp mate.cpp buscar.cpp
OBJS     = $(SRCS:.cpp=.o)

$(TARGET): $(OBJS)
 $(CXX) $(OBJS) -o $(TARGET)

%.o: %.cpp
 $(CXX) $(CXXFLAGS) -c $< -o $@

clean:
 rm -f $(OBJS) $(TARGET)
```

La notación `$(SRCS:.cpp=.o)` le pide a Make que sustituya `.cpp` por `.o` en cada nombre de `SRCS`: es equivalente a escribir `OBJS = main.o auxiliar.o ...` a mano.

CMake: multiplataforma y sin repetición

• ¿Por qué CMake si ya tenemos Make?

Make funciona perfectamente en Linux y Mac. Pero en Windows la sintaxis de las terminales es diferente (`rm` no existe, las rutas usan \ en lugar de /), y los Makefiles escritos en Linux no funcionan ahí sin modificaciones.

CMake resuelve esto: describes tu proyecto una sola vez en un archivo llamado `CMakeLists.txt` y CMake genera, en cada plataforma, el sistema de compilación nativo.

Plataforma	CMake genera
Linux / Mac	`Makefile`
Windows	Proyecto de Visual Studio (`.sln`)
Cualquiera (opcional)	Archivos Ninja (más rápido que Make)

• Un `CMakeLists.txt` mínimo

```
cmake_minimum_required(VERSION 3.10)

project(MiProyecto VERSION 1.0)

set(CMAKE_CXX_STANDARD 17)
```

```
set(CMAKE_CXX_STANDARD_REQUIRED True)

add_executable(programa
    main.cpp
    auxiliar.cpp
    obtener_datos.cpp
    mate.cpp
    buscar.cpp
)
```

Es más verboso que el Makefile, pero no tiene ninguna sintaxis dependiente del sistema operativo. El mismo `CMakeLists.txt` funciona en Linux, Mac y Windows sin cambios.

• Flujo de trabajo

CMake separa el código fuente de los archivos de compilación mediante un **build fuera de la fuente** (*out-of-source build*):

```
mkdir build   # crear directorio de compilación
cd build
cmake ..      # generar el Makefile (lee CMakeLists.txt del directorio padre)
make          # compilar
./programa    # ejecutar
```

Los archivos generados (`.o`, `Makefile`, etc.) quedan en `build/` y **no se suben al repositorio**. Para empezar de cero basta con borrar la carpeta `build/` y repetir el proceso.

> Agrega `build/` a tu `.gitignore` para que git nunca lo rastree. Solo `CMakeLists.txt` y los archivos fuente deben vivir en el repositorio.

Proyecto ejemplo: sistema de personas

El directorio `materialInicial/sesion19/proyecto1/` tiene un proyecto real con siete archivos. La siguiente tabla muestra qué responsabilidad tiene cada módulo:

Módulo	Responsabilidad
`persona.h`	Define la estructura `Persona` (nombre, apellidos, edad, salario). Sin lógica.
`auxiliar.h/.cpp`	Genera nombres y apellidos aleatorios de listas predefinidas.
`obtener_datos.h/.cpp`	Construye un arreglo de `Persona` con datos generados.
`mate.h/.cpp`	Calcula promedios de edad y salario.
`buscar.h/.cpp`	Ordena y filtra arreglos de `Persona`.
`main.cpp`	Orquesta todo: genera datos, calcula y muestra resultados.

El grafo de inclusión muestra por qué `persona.h` está en el centro: todos los módulos trabajan con el tipo `Persona`, así que todos lo incluyen.

```
            main.cpp
           /   |   \
  buscar.h  mate.h  obtener_datos.h
     |        |          |
persona.h persona.h  persona.h
                        |
                    auxiliar.h
```

Los guards de inclusión en `persona.h` garantizan que, aunque aparezca cuatro veces en la cadena de inclusiones, el preprocesador solo la procese una vez.

Ejercicios

1. Dado el proyecto mínimo (`main.cpp`, `funciones.h`, `funciones.cpp`), escribe el `Makefile` más simple posible: una sola regla con la línea de compilación directa (sin variables). Luego evoluciona ese Makefile agregando `CXX`, `CXXFLAGS`, `TARGET` y `SRCS` paso a paso.

2. Con el Makefile final de la sección anterior (usando `OBJS` y la regla patrón), modifica `funciones.cpp` y ejecuta `make`. ¿Qué archivos recompila y cuáles omite? Repite el experimento modificando solo `funciones.h`. ¿Cambia el resultado?

3. ¿Qué ocurre si defines una función dos veces en el mismo `.cpp`? ¿Y si la defines en dos `.cpp` distintos que se enlazan juntos? Prueba ambos casos y explica el mensaje de error en cada uno.

4. Crea un nuevo módulo `estadisticas.h` / `estadisticas.cpp` para el proyecto de personas con la función `double salario_maximo(Persona arr[], int n)`. Incluye el guard de inclusión correcto, agrégalo a `SRCS` en el Makefile, y escribe la línea equivalente en `CMakeLists.txt`.

5. ¿Cuál es la diferencia entre `make` y `make clean`? ¿Cuándo conviene ejecutar `clean` antes de recompilar?

Resumen

- Dividir el código en módulos (`.h` + `.cpp`) mejora la legibilidad, la colaboración y la velocidad de compilación.
- El **archivo de cabecera** contiene declaraciones (el *qué*); el **archivo de implementación** contiene definiciones (el *cómo*).
- Los **guards de inclusión** (`#ifndef` / `#define` / `#endif` o `#pragma once`) evitan que el mismo `.h` se procese más de una vez.
- La compilación tiene dos pasos: **compilar** (`.cpp` → `.o`) y **enlazar** (`.o` → ejecutable).
- Separar los pasos permite recompilar solo los módulos que cambiaron.

- Un **Makefile** puede empezar tan simple como una sola regla con la línea de compilación, y evolucionar con variables (`CXX`, `CXXFLAGS`, `SRCS`, `OBJS`) conforme el proyecto crece.
- **Make** automatiza la recompilación: compara fechas de modificación y reconstruye solo lo que cambió.
- **CMake** genera el sistema de compilación adecuado para cada plataforma a partir de un `CMakeLists.txt` portátil.